

FreeRTOS 新手入门教学

从裸机思维到多任务实时系统

嵌入式学习资料

2026 年 9 月 2 日

目录

1	为什么需要 RTOS	2
2	核心概念	2
2.1	任务	2
2.2	调度器	3
2.3	Tick	3
2.4	任务状态	3
3	创建你的第一个任务	4
4	延时与周期任务	5
4.1	相对延时	5
4.2	绝对周期延时	5
5	队列	6
5.1	创建队列	6
5.2	发送和接收	6
6	信号量与互斥量	7
6.1	二值信号量	7

目录	2
6.2 计数信号量	7
6.3 互斥量	8
7 事件组	8
8 任务通知	9
9 软件定时器	10
10 中断与 FreeRTOS	11
11 常用配置项	11
12 内存管理	12
13 常见问题与排查	13
13.1 栈溢出	13
13.2 优先级反转	13
13.3 死锁	13
13.4 中断中调用错误 API	13
13.5 内存碎片	14
14 新手学习路线	14
14.1 先打好基础	14
14.2 再做小实验	14
14.3 完成一个完整项目	15
15 学习检查清单	15
16 总结	16

1 为什么需要 RTOS

在没有操作系统时，我们通常写一个超级循环：

```
1 int main(void)
2 {
3     init_hardware();
4
5     while (1) {
6         read_sensor();
7         process_data();
8         update_display();
9         handle_communication();
10    }
11 }
```

这种写法对小项目足够，但任务变复杂后会出现问题：

- 某个函数执行时间过长，其他功能无法及时响应。
- 通信、按键、显示、传感器之间互相等待。
- 阻塞操作会导致整个系统停顿。
- 实时性要求无法保证。

RTOS 把复杂系统拆成多个“任务”，由调度器决定哪个任务运行。FreeRTOS 是最常用的 MCU RTOS 之一，支持 STM32、ESP32、NXP、TI、Nordic 等平台。

2 核心概念

2.1 任务

任务就是一个独立执行的函数。它有自己的栈空间、优先级和运行状态。

```
1 void vMyTask(void *pvParameters)
2 {
3     while (1) {
4         do_something();
5         vTaskDelay(pdMS_TO_TICKS(100));
6     }
```

```
7 }
```

2.2 调度器

调度器负责决定当前应该运行哪个任务。FreeRTOS 通常使用抢占式调度：

- 高优先级任务就绪后，会抢占低优先级任务。
- 相同优先级的任务可以时间片轮转运行。
- 任务主动阻塞或延时，调度器会切换到其他任务。

2.3 Tick

Tick 是 FreeRTOS 的时间基准，通常由硬件定时器产生。

```
1 #define configTICK_RATE_HZ 1000
```

如果 Tick 频率是 1000Hz，那么：

- 1 个 Tick 等于 1ms。
- `pdMS_TO_TICKS(100)` 会把 100ms 转换为 100 个 Tick。

2.4 任务状态

FreeRTOS 任务通常有以下状态：

- **运行态 Running**：任务正在使用 CPU。
- **就绪态 Ready**：任务可以运行，但在等待 CPU。
- **阻塞态 Blocked**：任务在等待延时、队列、信号量或事件。
- **挂起态 Suspended**：任务被主动挂起，不参与调度。

任务通常从就绪态进入运行态；调用延时或等待 API 时进入阻塞态；等待完成后回到就绪态。

3 创建你的第一个任务

FreeRTOS 创建任务最常用的 API 是：

```
1 BaseType_t xTaskCreate(  
2     TaskFunction_t pxTaskCode,  
3     const char *pcName,  
4     configSTACK_DEPTH_TYPE usStackDepth,  
5     void *pvParameters,  
6     UBaseType_t uxPriority,  
7     TaskHandle_t *pxCreatedTask  
8 );
```

参数含义：

- pxTaskCode: 任务函数。
- pcName: 任务名称，便于调试。
- usStackDepth: 栈深度，注意单位不是字节。
- pvParameters: 传给任务的参数。
- uxPriority: 任务优先级，数值越大优先级越高。
- pxCreatedTask: 任务句柄，可传 NULL。

完整示例：

```
1 #include "FreeRTOS.h"  
2 #include "task.h"  
3  
4 void vTask1(void *pvParameters)  
5 {  
6     while (1) {  
7         printf("Task 1 running\n");  
8         vTaskDelay(pdMS_TO_TICKS(1000));  
9     }  
10 }  
11  
12 void vTask2(void *pvParameters)  
13 {  
14     while (1) {
```

```
15     printf("Task 2 running\n");
16     vTaskDelay(pdMS_TO_TICKS(2000));
17 }
18 }
19
20 int main(void)
21 {
22     xTaskCreate(vTask1, "Task1", 2048, NULL, 1, NULL);
23     xTaskCreate(vTask2, "Task2", 2048, NULL, 1, NULL);
24
25     vTaskStartScheduler();
26
27     for (;;) {
28         // The scheduler should never return here.
29     }
30 }
```

4 延时与周期任务

4.1 相对延时

`vTaskDelay` 会让当前任务阻塞指定时间：

```
1 vTaskDelay(pdMS_TO_TICKS(500));
```

它表示从调用时刻开始，再延时 500ms。

4.2 绝对周期延时

如果希望任务保持固定周期，应使用：

```
1 void vPeriodicTask(void *pvParameters)
2 {
3     TickType_t xLastWakeTime = xTaskGetTickCount();
4
5     while (1) {
6         do_periodic_work();
7
8         vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(100));
```

```
9     }  
10 }
```

这种写法能减少任务执行时间造成的周期漂移。

5 队列

队列用于在任务之间传递数据。

5.1 创建队列

```
1 QueueHandle_t xQueue = xQueueCreate(5, sizeof(int));
```

这里创建了一个最多存放 5 个 int 的队列。

5.2 发送和接收

```
1 void vProducer(void *pvParameters)  
2 {  
3     int value = 0;  
4  
5     while (1) {  
6         xQueueSend(xQueue, &value, portMAX_DELAY);  
7         value++;  
8         vTaskDelay(pdMS_TO_TICKS(100));  
9     }  
10 }  
11  
12 void vConsumer(void *pvParameters)  
13 {  
14     int value;  
15  
16     while (1) {  
17         if (xQueueReceive(xQueue, &value, portMAX_DELAY) == pdTRUE) {  
18             process_value(value);  
19         }  
20     }  
21 }
```

队列的行为：

- 队列满时，发送任务会阻塞。
- 队列空时，接收任务会阻塞。
- 超时参数 `portMAX_DELAY` 表示无限等待。

队列适合传递数据，但相对任务通知和信号量开销稍大。

6 信号量与互斥量

6.1 二值信号量

二值信号量常用于“等待一个事件发生”。

```
1 SemaphoreHandle_t xSem = xSemaphoreCreateBinary();
2
3 void vWaitingTask(void *pvParameters)
4 {
5     while (1) {
6         if (xSemaphoreTake(xSem, portMAX_DELAY) == pdTRUE) {
7             handle_event();
8         }
9     }
10 }
11
12 void vSignalingTask(void *pvParameters)
13 {
14     while (1) {
15         do_work();
16         xSemaphoreGive(xSem);
17         vTaskDelay(pdMS_TO_TICKS(500));
18     }
19 }
```

6.2 计数信号量

计数信号量可以表示多个资源或多次事件：


```
1 SemaphoreHandle_t xCountingSem =  
2     xSemaphoreCreateCounting(3, 0);
```

参数分别表示最大计数和初始计数。

6.3 互斥量

互斥量用于保护共享资源。它是带所有权的特殊二值信号量。

```
1 SemaphoreHandle_t xMutex = xSemaphoreCreateMutex();  
2  
3 void vAccessResource(void *pvParameters)  
4 {  
5     while (1) {  
6         if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {  
7             access_shared_resource();  
8             xSemaphoreGive(xMutex);  
9         }  
10    }  
11 }
```

互斥量的重要规则：

- 谁 Take，谁 Give。
- 支持优先级继承。
- 不能在中断服务程序中使用。
- 同一个任务多次获取同一互斥量时，应使用递归互斥量。

7 事件组

事件组可以一次等待多个事件。

```
1 #define BIT_TEMP_READY (1 << 0)  
2 #define BIT_WIFI_READY (1 << 1)  
3  
4 EventGroupHandle_t xEvents = xEventGroupCreate();  
5
```

```
6 void vSensorTask(void *pvParameters)
7 {
8     read_temperature();
9     xEventGroupSetBits(xEvents, BIT_TEMP_READY);
10 }
11
12 void vWiFiTask(void *pvParameters)
13 {
14     connect_wifi();
15     xEventGroupSetBits(xEvents, BIT_WIFI_READY);
16 }
17
18 void vMainTask(void *pvParameters)
19 {
20     xEventGroupWaitBits(
21         xEvents,
22         BIT_TEMP_READY | BIT_WIFI_READY,
23         pdTRUE,
24         pdTRUE,
25         portMAX_DELAY
26     );
27
28     start_application();
29 }
```

这里 `pdTRUE` 表示等待到所有事件位都被置位，并在等待成功后清除这些事件位。

8 任务通知

任务通知是轻量级的任务间通信方式。相比队列和信号量，它通常更快，占用的 RAM 更少。

```
1 void vProducerTask(void *pvParameters)
2 {
3     while (1) {
4         do_something();
5         xTaskNotifyGive(xConsumerHandle);
6         vTaskDelay(pdMS_TO_TICKS(100));
7     }
8 }
```

```
9
10 void vConsumerTask(void *pvParameters)
11 {
12     while (1) {
13         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
14         handle_notification();
15     }
16 }
```

任务通知的限制：

- 通知对象只能是一个任务。
- 一个任务只有一个通知值。
- 适合简单事件通知，不适合复杂的多数据流。

9 软件定时器

软件定时器由 FreeRTOS 的定时器服务任务执行，因此回调函数不能长时间阻塞。

```
1 void vTimerCallback(TimerHandle_t xTimer)
2 {
3     toggle_led();
4 }
5
6 TimerHandle_t xTimer = xTimerCreate(
7     "Blink",
8     pdMS_TO_TICKS(1000),
9     pdTRUE,
10    NULL,
11    vTimerCallback
12 );
13
14 xTimerStart(xTimer, 0);
```

参数含义：

- 定时器名称。
- 周期时间。

- 是否自动重载。
- 定时器 ID。
- 回调函数。

10 中断与 FreeRTOS

中断里调用 FreeRTOS API 时，必须使用带 FromISR 的接口。

```
1 void IRAM_ATTR gpio_isr_handler(void *arg)
2 {
3     BaseType_t xHigherPriorityTaskWoken = pdFALSE;
4
5     xSemaphoreGiveFromISR(xSem, &xHigherPriorityTaskWoken);
6
7     portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
8 }
```

重要规则：

- 不要在中断中调用 xSemaphoreGive，应调用 xSemaphoreGiveFromISR。
- 不要在中断中调用互斥量 Take/Give。
- portYIELD_FROM_ISR 用于判断是否需要在退出中断后立即切换任务。
- 中断优先级必须正确配置，尤其是 Cortex-M 上使用 FreeRTOS 时。

11 常用配置项

FreeRTOS 的行为主要由 FreeRTOSConfig.h 控制。

```
1 #define configUSE_PREEMPTION          1
2 #define configTICK_RATE_HZ           1000
3 #define configMAX_PRIORITIES          8
4 #define configMINIMAL_STACK_SIZE      128
5 #define configTOTAL_HEAP_SIZE         (32 * 1024)
6 #define configUSE_IDLE_HOOK           0
7 #define configUSE_TICK_HOOK           0
8 #define configUSE_MUTEXES             1
```

```
9 #define configUSE_RECURSIVE_MUTEXES      1
10 #define configUSE_COUNTING_SEMAPHORES    1
11 #define configSUPPORT_STATIC_ALLOCATION    1
12 #define configSUPPORT_DYNAMIC_ALLOCATION  1
```

重点配置：

- configUSE_PREEMPTION：是否使用抢占式调度。
- configTICK_RATE_HZ：Tick 频率。
- configMAX_PRIORITIES：最大优先级数量。
- configTOTAL_HEAP_SIZE：FreeRTOS 动态内存池大小。

12 内存管理

FreeRTOS 提供多种堆实现：

- heap_1：只分配，不释放，适合永远不删除对象的系统。
- heap_2：可以释放，但不合并相邻空闲块。
- heap_3：使用标准库 malloc/free。
- heap_4：支持释放并合并空闲块，是常见选择。
- heap_5：允许跨多个不连续内存区域。

动态创建：

```
1 xTaskCreate(...);
2 xQueueCreate(...);
3 xSemaphoreCreateBinary();
```

静态创建：

```
1 xTaskCreateStatic(...);
2 xQueueCreateStatic(...);
3 xSemaphoreCreateBinaryStatic(...);
```

静态创建不依赖动态堆，适合对内存使用要求非常严格的系统。

13 常见问题与排查

13.1 栈溢出

任务栈过小时会发生栈溢出，可能表现为随机复位或数据被破坏。

排查方法：

- 增大任务栈。
- 使用 `uxTaskGetStackHighWaterMark` 查看剩余栈。
- 开启栈溢出检测钩子。

13.2 优先级反转

高优先级任务等待低优先级任务持有的互斥量，而低优先级任务又被中优先级任务抢占。

FreeRTOS 的互斥量支持优先级继承，应优先使用互斥量而不是普通二值信号量来保护共享资源。

13.3 死锁

两个任务互相等待对方释放资源，导致谁也无法继续。

解决方法：

- 固定加锁顺序。
- 使用超时等待。
- 减少嵌套锁。

13.4 中断中调用错误 API

在中断里调用非 `FromISR` API 会导致未定义行为。中断中应使用：

```
1 xQueueSendFromISR(...);  
2 xSemaphoreGiveFromISR(...);  
3 xTaskNotifyFromISR(...);
```

13.5 内存碎片

频繁创建和删除不同大小的对象可能产生内存碎片。建议：

- 使用 `heap_4`。
- 尽量在系统启动阶段创建对象。
- 对关键对象使用静态创建。

14 新手学习路线

14.1 先打好基础

- C 语言：指针、结构体、函数指针、位运算、环形缓冲区。
- 单片机外设：GPIO、UART、SPI、I2C、定时器、ADC、DMA。
- 中断：中断优先级、ISR、清除中断标志。

14.2 再做小实验

按顺序练习：

1. 创建两个任务，用不同频率打印信息。
2. 用一个任务读取按键，另一个任务控制 LED。
3. 使用队列在任务之间传递传感器数据。
4. 使用二值信号量处理外部中断事件。
5. 使用互斥量保护共享变量。
6. 使用事件组等待多个条件。
7. 使用软件定时器周期性执行任务。

14.3 完成一个完整项目

推荐做：

- 多传感器采集与显示设备。
- WiFi 或 BLE 物联网节点。
- 带 OTA 升级的低功耗设备。
- 电机或舵机控制设备。
- CAN 总线工业节点。

完整项目比多个零散 demo 更能证明能力。

15 学习检查清单

完成入门学习后，你应该能回答：

- FreeRTOS 的任务有哪几种状态？
- `vTaskDelay` 和 `vTaskDelayUntil` 有什么区别？
- 队列、二值信号量、计数信号量、互斥量分别适合什么场景？
- 任务通知和队列相比有什么优缺点？
- 为什么中断里要使用 `FromISR` API？
- 什么是优先级反转？如何避免？
- 什么是死锁？如何避免？
- 如何判断任务栈是否足够？
- `heap_1` 到 `heap_5` 有什么区别？

16 总结

FreeRTOS 学习的重点不是背 API，而是建立多任务系统思维：

- 如何把功能拆成任务。
- 如何设置优先级。
- 如何选择任务间通信方式。
- 如何保证共享资源安全。
- 如何调试实时性问题。

建议先在一款开发板上完成一个完整项目，再逐步深入源码、调度器和性能分析工具。